

INSTITUTO INFNET

Projeto de Bloco: Sistemas Roboticos — 2026.2

## TERMO DE ABERTURA DE PROJETO (TAP)

### **Projeto ARACNE**

*Aplicação de Reconhecimento de Aracnídeos por Câmera em Nós Embarcados*

Trabalho de Projeto — TP1 a TP5

Aluno: [SEU NOME COMPLETO]

Professor: Dacio Moreira de Souza

Rio de Janeiro — Agosto de 2026

## 1. Identificação do Projeto

| Campo                  | Descrição                                                                                                          |
|------------------------|--------------------------------------------------------------------------------------------------------------------|
| Nome do projeto        | ARACNE — Aplicação de Reconhecimento de Aracnídeos por Câmera em Nós Embarcados                                    |
| Aluno responsável      | [seu nome completo] — matrícula [xxxxx]                                                                            |
| Instituição            | Instituto Infnet                                                                                                   |
| Disciplina             | Projeto de Bloco: Sistemas Roboticos — 2026.2                                                                      |
| Professor orientador   | Dacio Moreira de Souza                                                                                             |
| Data de abertura       | 28/08/2026                                                                                                         |
| Ciclo de entregas      | TP1 28/08 - TP2 25/09 - TP3 23/10 - TP4 21/11 - TP5 27/11 - Final 04/12                                            |
| Categoria do projeto   | Projeto próprio, derivado da sugestão "robô perceptivo com câmera e/ou sensores", sujeito a aprovação do professor |
| Versão deste documento | v1.0 — [dd/mm/aaaa]                                                                                                |

## 2. Contexto e Justificativa

O Brasil concentra uma das maiores diversidades de aracnídeos do mundo e, ao mesmo tempo, registra milhares de acidentes anuais envolvendo aranhas de importância médica. Grande parte da gravidade desses acidentes não decorre da picada em si, mas do atraso ou do erro na identificação do animal: o atendimento correto depende de saber, com rapidez, a que gênero pertence o exemplar encontrado. A identificação, hoje, depende de conhecimento especializado em aracnologia, que não está disponível no momento e no local em que o encontro ocorre.

Esse é um problema de percepção visual em ambiente não controlado — exatamente a classe de problema para a qual sistemas robóticos perceptivos são projetados. Um agente que observa o mundo por uma câmera, segmenta a região de interesse, extrai características e classifica o que vê reproduz, em escala reduzida, o pipeline de percepção de um robô de inspeção real.

A escolha do ROS 2 como espinha dorsal não é decorativa. O problema é naturalmente distribuído: aquisição de imagem, pré-processamento, segmentação, extração de features e inferência são etapas com taxas de execução, custos computacionais e ciclos de vida diferentes. Modelá-las como nós independentes que se comunicam por tópicos e serviços permite (i) substituir a fonte de imagem — webcam, vídeo gravado ou câmera da Raspberry Pi — sem alterar o restante do sistema; (ii) instrumentar e depurar cada etapa isoladamente com as ferramentas nativas (ros2 topic, rqt\_graph, rqt\_image\_view); e (iii) distribuir o processamento entre máquinas via DDS, caso o módulo de inferência precise migrar para outro host. Uma implementação monolítica em um único script Python não ofereceria nenhuma dessas propriedades.

Do ponto de vista acadêmico, o projeto exercita integralmente o conteúdo previsto para os TPs: comunicação publisher/subscriber, serviços, conversão de espaços de cor, segmentação, extração de features, análise do grafo de comunicação e documentação de arquitetura.

## 3. Objetivos

### 3.1. Objetivo geral

Desenvolver um sistema perceptivo baseado em ROS 2 capaz de capturar imagens de uma câmera em tempo real, segmentar e classificar aracnídeos entre quatro espécies previamente definidas e retornar a classificação taxonômica completa (reino, filo, subfilo, classe, ordem, família, gênero e espécie), o nome popular e as características descritivas do exemplar identificado.

### 3.2. Objetivos específicos

- Configurar e validar um ambiente ROS 2 Humble sobre Ubuntu 22.04 em máquina virtual, com workspace Colcon e OpenCV integrados.
- Implementar um nó publisher que capture frames da webcam e os publique em /camera/image\_raw no formato sensor\_msgs/Image.
- Implementar um nó subscriber que aplique conversão BGR → HSV e segmentação por cor, publicando a máscara resultante e a contagem de objetos segmentados.
- Disponibilizar o serviço /vision/status, que retorna sob demanda o número de objetos segmentados no último frame processado.
- Implementar o pipeline de visão computacional com detecção de faces (Haar Cascade) e extração de features (ORB/HOG) sobre os frames recebidos.
- Construir um dataset de no mínimo 300 imagens por espécie, a partir de acervos públicos de licença aberta complementados por imagens próprias.
- Treinar um classificador por transfer learning e integrá-lo como nó de inferência ROS 2, com resposta em tempo aceitável para operação interativa.
- Documentar a arquitetura implementada com diagrama de nós e tópicos e análise do grafo de comunicação gerado pelo rqt\_graph.
- Entregar relatório em PDF, repositório GitHub versionado e vídeo de defesa de até 5 minutos.

## 4. Descrição do Produto e Funcionalidades Previstas

O produto final é um conjunto de pacotes ROS 2 que, executados em conjunto, formam um sistema de identificação de aracnídeos assistido por câmera. O usuário posiciona o exemplar (ou uma imagem dele) diante da webcam; o sistema exibe, em tempo real, a máscara de segmentação, as features extraídas e um painel com a identificação taxonômica e o grau de confiança da predição.

| #  | Funcionalidade               | Descrição                                                                                                                              |
|----|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| F1 | Aquisição contínua de imagem | Captura de frames da webcam Logitech 720p a uma taxa configurável e publicação em tópico ROS 2 padronizado.                            |
| F2 | Segmentação por cor em HSV   | Conversão do espaço de cor e aplicação de máscaras com limiares ajustáveis, com operações morfológicas para redução de ruído.          |
| F3 | Contagem de objetos          | Detecção de contornos na máscara segmentada, com filtro por área mínima, e publicação da contagem.                                     |
| F4 | Serviço de status da visão   | Serviço ROS 2 que retorna, sob requisição, o número de objetos segmentados no último frame e o instante do processamento.              |
| F5 | Detecção de faces            | Detecção de faces humanas por classificador em cascata, atendendo ao requisito da disciplina e servindo de baseline comparativo para o |

| #   | Funcionalidade              | Descrição                                                                                                                        |
|-----|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
|     |                             | detector de aracnídeos.                                                                                                          |
| F6  | Extração de features        | Extração de pontos de interesse e descritores (ORB) e de histogramas de gradientes orientados (HOG) sobre a região de interesse. |
| F7  | Classificação de espécie    | Inferência sobre a ROI segmentada, retornando a espécie mais provável entre as quatro do escopo e o grau de confiança.           |
| F8  | Resolução taxonômica        | A partir da espécie predita, resolução da linhagem taxonômica completa, nome popular e características descritivas.              |
| F9  | Visualização e overlay      | Sobreposição das informações sobre o frame original e publicação em tópico de imagem anotada.                                    |
| F10 | Introspecção da arquitetura | Geração do grafo de comunicação para análise e documentação da arquitetura.                                                      |

## 5. Escopo

### 5.1. Está no escopo

- Reconhecimento de quatro espécies de aracnídeos, listadas na seção 7.4, mais uma classe negativa ("ausência de aracnídeo") para reduzir falsos positivos.
- Operação sobre imagem de webcam USB em tempo real e sobre arquivos de vídeo e imagens estáticas.
- Execução completa em máquina virtual Ubuntu 22.04 com ROS 2 Humble.
- Treinamento do modelo em ambiente externo (nuvem) e execução da inferência localmente na VM.
- Documentação, versionamento e vídeo de defesa conforme exigido pelo Moodle.

### 5.2. Está fora do escopo

A delimitação explícita do que não será feito é parte do compromisso técnico deste TAP:

- Identificação de espécies fora do conjunto definido — exemplares não pertencentes às quatro classes devem ser rejeitados por baixa confiança, não classificados forçadamente.
- Qualquer forma de aconselhamento médico, diagnóstico de acidente ou recomendação de tratamento. O sistema é uma ferramenta de identificação visual de caráter acadêmico e educacional, e essa limitação será declarada na interface e no relatório.
- Determinação de sexo, estágio de desenvolvimento ou medidas biométricas do exemplar.
- Manipulação física de exemplares vivos. O conjunto de imagens será construído a partir de acervos públicos e de fotografias de exemplares observados a distância segura, sem captura ou contenção de animais.
- Navegação autônoma, atuação mecânica ou controle de motores.
- Publicação do sistema como aplicativo, serviço web ou produto para uso por terceiros.

## 6. Requisitos

### 6.1. Requisitos funcionais

| ID   | Requisito                                                                                                              | Prioridade  |
|------|------------------------------------------------------------------------------------------------------------------------|-------------|
| RF01 | O sistema deve publicar frames capturados da câmera no tópico /camera/image_raw.                                       | Obrigatório |
| RF02 | O sistema deve converter os frames de BGR para HSV e aplicar segmentação por faixa de cor.                             | Obrigatório |
| RF03 | O sistema deve expor o serviço /vision/status retornando a contagem de objetos do último frame.                        | Obrigatório |
| RF04 | O sistema deve aplicar detecção de faces sobre os frames recebidos.                                                    | Obrigatório |
| RF05 | O sistema deve extrair features dos frames e disponibilizá-las para inspeção.                                          | Obrigatório |
| RF06 | O sistema deve classificar a imagem entre as quatro espécies do escopo e a classe negativa.                            | Obrigatório |
| RF07 | O sistema deve retornar reino, filo, subfilo, classe, ordem, família, gênero, espécie, nome popular e características. | Obrigatório |
| RF08 | O sistema deve informar o grau de confiança da predição e sinalizar predições abaixo do limiar.                        | Obrigatório |
| RF09 | O sistema deve publicar um frame anotado com o resultado da identificação.                                             | Desejável   |
| RF10 | O sistema deve permitir a substituição da fonte de imagem por arquivo de vídeo via parâmetro, sem alteração de código. | Desejável   |

### 6.2. Requisitos não funcionais

| ID    | Requisito                                                                                   | Critério                   |
|-------|---------------------------------------------------------------------------------------------|----------------------------|
| RNF01 | A aquisição e publicação de imagem deve sustentar taxa mínima de operação interativa.       | $\geq 10$ FPS a 640×480    |
| RNF02 | A inferência de classificação deve responder em tempo compatível com uso interativo em CPU. | $\leq 500$ ms por frame    |
| RNF03 | O sistema deve executar integralmente dentro da VM com recursos limitados.                  | 4 GB RAM, 2 vCPU           |
| RNF04 | O código deve estar versionado com histórico de commits que evidencie a evolução por TP.    | Repositório GitHub público |
| RNF05 | Cada etapa de configuração e execução deve estar documentada com evidência visual.          | Capturas de tela por etapa |
| RNF06 | A arquitetura deve ser modular, com nós independentes e responsabilidade única.             | Grafo com nós desacoplados |
| RNF07 | O dataset deve utilizar apenas imagens com licença que permita uso                          | Licenças CC                |

| ID | Requisito  | Critério   |
|----|------------|------------|
|    | acadêmico. | declaradas |

## 7. Arquitetura Técnica Proposta

O sistema é decomposto em cinco nós ROS 2 com responsabilidade única, comunicando-se por tópicos e um serviço. Essa decomposição é o coração do projeto: cada nó pode ser executado, testado e substituído isoladamente.

### 7.1. Nós, tópicos e serviços

| Nó                | Tipo                            | Entrada                                     | Saída                                                       |
|-------------------|---------------------------------|---------------------------------------------|-------------------------------------------------------------|
| camera_node       | Publisher                       | Dispositivo /dev/video0 ou arquivo de vídeo | /camera/image_raw (sensor_msgs/Image)                       |
| segmentation_node | Subscriber + Publisher + Server | /camera/image_raw                           | /vision/mask, /vision/object_count e serviço /vision/status |
| features_node     | Subscriber + Publisher          | /camera/image_raw                           | /vision/faces, /vision/features_image                       |
| classifier_node   | Subscriber + Publisher          | /camera/image_raw, /vision/mask             | /aracne/identification (mensagem customizada)               |
| overlay_node      | Subscriber + Publisher          | /camera/image_raw, /aracne/identification   | /aracne/annotated_image                                     |

### 7.2. Interfaces customizadas

Será criado o pacote `aracne_interfaces` contendo a mensagem `SpiderID.msg` (campos: reino, filo, subfilo, classe, ordem, família, genero, especie, nome\_popular, características, confiança, timestamp) e o serviço `VisionStatus.srv` (resposta: object\_count, last\_frame\_stamp, ok). A definição de interfaces próprias, em vez do uso de mensagens genéricas do tipo `String`, é o que torna o sistema legível para outros nós e demonstra domínio do modelo de dados do ROS 2.

### 7.3. Estratégia de classificação e resolução taxonômica

Decisão de projeto relevante: o modelo de visão computacional não prediz a taxonomia completa. Ele prediz apenas a espécie. A linhagem taxonômica é determinística — dada a espécie, reino, filo, subfilo, classe, ordem, família e gênero são consequência lógica, não inferência. Fazer a rede aprender oito saídas correlacionadas seria desperdício de capacidade e fonte de inconsistência (o modelo poderia prever uma família incompatível com o gênero).

A arquitetura adotada é, portanto: a rede neural produz uma distribuição de probabilidade sobre cinco classes (quatro espécies e uma classe negativa); a espécie de maior probabilidade, se acima do limiar de confiança, é usada como chave em um arquivo `taxonomy.json` que devolve a linhagem completa, o nome popular e as características descritivas. Esse desacoplamento entre percepção e conhecimento simbólico é padrão em sistemas robóticos reais e será justificado no relatório.

O classificador será construído por transfer learning sobre uma rede pré-treinada leve (MobileNetV3-Small ou EfficientNet-B0), com congelamento do backbone e retreinamento da cabeça de classificação. O modelo será exportado em formato ONNX e executado na VM via onnxruntime, em CPU.

## 7.4. Espécies do escopo inicial

A seleção prioriza espécies brasileiras com boa disponibilidade de imagens em acervos públicos e, sobretudo, com separabilidade visual entre si — o que torna o problema tratável com um dataset pequeno. Todas compartilham a linhagem até a classe Arachnida, divergindo a partir da família:

| Espécie                          | Nome popular           | Família       | Traço visual dominante                              |
|----------------------------------|------------------------|---------------|-----------------------------------------------------|
| <i>Latrodectus curacaviensis</i> | Viúva-negra            | Theridiidae   | Abdome globoso escuro com marcas vermelhas          |
| <i>Trichonephila clavipes</i>    | Aranha-de-teia-dourada | Araneidae     | Corpo alongado, pernas com tufo, seda dourada       |
| <i>Lycosa erythrognatha</i>      | Aranha-de-grama        | Lycosidae     | Corpo robusto acinzentado, quelíceras avermelhadas  |
| <i>Avicularia avicularia</i>     | Caranguejeira-rosa     | Theraphosidae | Porte grande, pilosidade densa, extremidades claras |

*Linhagem comum a todas: Reino Animalia → Filo Arthropoda → Subfilo Chelicerata → Classe Arachnida → Ordem Araneae.*

## 8. Stack Tecnológica e Ambiente

| Camada                  | Tecnologia                                                                                            |
|-------------------------|-------------------------------------------------------------------------------------------------------|
| Host                    | Fedora Linux (instalação nativa), 8 GB RAM                                                            |
| Ambiente ROS 2          | Ubuntu 22.04 LTS (VirtualBox com a ISO fornecida, ou container oficial ROS 2 — ver docs/decisoões.md) |
| Versionamento e entrega | GitHub Classroom (branch dev, main, entrega-tpN e tag tpN) + Moodle                                   |
| Middleware robótico     | ROS 2 Humble Hawksbill                                                                                |
| Build system            | Colcon, com workspace ~/aracne_ws                                                                     |
| Visão computacional     | OpenCV 4 (Python), cv_bridge                                                                          |
| Aprendizado de máquina  | PyTorch para treinamento (nuvem); ONNX Runtime para inferência (VM)                                   |
| Linguagem               | Python 3.10 (rclpy)                                                                                   |
| Camera                  | Logitech 720p USB, com fallback automatico para fonte sintetica                                       |
| Hardware opcional       | Raspberry Pi 3 — avaliação de viabilidade como nó remoto de aquisição                                 |
| Pacote ROS 2            | percepcao_aracnideos (ament_python)                                                                   |
| Gravação da defesa      | OBS Studio ou Loom                                                                                    |

Justificativa da divisão treinamento/inferência: com 8 GB de RAM no host e 4 GB alocados à VM, treinar uma rede convolucional localmente é inviável em tempo hábil. O treinamento será executado em ambiente de nuvem com GPU gratuita, e apenas o modelo exportado (poucos megabytes) retorna para a VM. Essa separação não é uma limitação contornada, mas a prática corrente em robótica: treina-se offline, embarca-se o modelo inferido.

## 9. Planejamento das Entregas (TP1 a TP5)

O planejamento abaixo segue o calendario e os gates oficiais da disciplina. O reconhecimento de aracnideos e o dominio de percepcao do projeto: ele fornece as classes que o sistema enxerga, enquanto a espinha robotica (interfaces, acoes, mapeamento, navegacao e manipulacao) evolui por cima dele a cada TP.

| TP           | Entrega            | Escopo e aplicacao ao dominio                                                                                                                                                                                                                                                                                                                                      |
|--------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TP1</b>   | 28/08/2026         | Ambiente, comunicacao basica e pipeline inicial de visao. camera_node publica em /camera/image_raw; vision_node converte para HSV, segmenta por cor, conta alvos e serve /vision/status. Gates G1.0 a G1.6.                                                                                                                                                        |
| <b>TP2</b>   | 25/09/2026         | Interfaces proprias, acoes e parametrizacao. Pacote percepcao_aracnideos_interfaces com SpiderID.msg e VisionStatus.srv; action de varredura com feedback periodico e cancelamento; deteccao evoluida com metrica declarada; parametros em YAML e URDF no RViz2. Gates G2.0 a G2.6.                                                                                |
| <b>TP3</b>   | 23/10/2026         | Integracao, mapeamento e percepcao avancada. bringup.launch.py subindo o sistema inteiro; cenario em Gazebo; arvore TF completa; SLAM Toolbox com mapa persistido; segmentacao por instancia publicando as especies em /vision/segmented. Gates G3.0 a G3.6.                                                                                                       |
| <b>TP4</b>   | 21/11/2026 ate 12h | Navegacao autonoma, registro e percepcao veicular. Nav2 com o mapa do TP3; navegacao por waypoints; comportamento de obstaculo; ros2 bag reproduzivel; percepcao veicular no MetaDrive; CNN treinada com no minimo 500 frames, comparacao com e sem augmentation e matriz de confusao — e aqui que o classificador de especies entra em regime. Gates G4.0 a G4.7. |
| <b>TP5</b>   | 27/11/2026         | Manipulacao, aprendizado e validacao. MoveIt 2 planejando para tres poses; dataset de mil frames; baseline PID medido; Behavioral Cloning comparado ao PID; PPO no highway-env. Gates G5.0 a G5.7.                                                                                                                                                                 |
| <b>Final</b> | 04/12/2026         | Sistema integrado subindo com um comando em maquina limpa; video publicado; repositório navegavel; relatorio consolidado. Gates GF.0 a GF.5.                                                                                                                                                                                                                       |
| <b>Banca</b> | 08 e 15/12/2026    | Apresentacao e arguicao, com plano B de demo gravada.                                                                                                                                                                                                                                                                                                              |

*Atencao ao intervalo entre o TP4 (21/11) e o TP5 (27/11): sao seis dias. O gate G5.0 tem data anterior a entrega do TP4 justamente por isso — o TP5 precisa comecar durante o TP4. O treinamento do PPO, em particular, roda em segundo plano por horas e deve ser disparado cedo.*



## 10. Cronograma e Disponibilidade

A disponibilidade concentra-se fora do horario de trabalho (10h00 as 16h30, dias uteis), resultando em janelas de cerca de tres horas em noites de dias uteis e blocos maiores aos fins de semana. Os gates da disciplina trazem datas recomendadas com folga em relacao a cada entrega, e o cronograma abaixo as adota.

| TP    | Janela        | Esforco estimado | Atividade critica                                      |
|-------|---------------|------------------|--------------------------------------------------------|
| TP1   | ate 28/08     | 10 a 14 h        | Montagem do ambiente — e o que trava se falhar         |
| TP2   | 29/08 a 25/09 | 25 a 30 h        | Action com cancelamento real, nao publisher disfarçado |
| TP3   | 26/09 a 23/10 | 35 a 45 h        | Arvore TF antes do SLAM; TF errada queima uma semana   |
| TP4   | 24/10 a 21/11 | 40 a 50 h        | Nav2 e a CNN; coleta de frames começa no TP3           |
| TP5   | 14/11 a 27/11 | 30 a 40 h        | PPO disparado cedo, rodando em segundo plano           |
| Final | 28/11 a 04/12 | 12 a 16 h        | Reproducao em maquina limpa                            |
| Banca | ate 15/12     | 6 a 8 h          | Ensaio cronometrado com demo gravada de reserva        |

Estrategia de paralelizacao: a construcao do dataset de aracnideos e a atividade de maior risco de atraso e menor dependencia tecnica das demais. Ela começa no TP2 e avanca em paralelo, para que o treinamento da CNN exigido no G4.6 nao fique represado ate novembro.

## 11. Riscos e Plano de Mitigação

| ID | Risco                                                                                              | Impacto | Mitigação                                                                                                                                           |
|----|----------------------------------------------------------------------------------------------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| R1 | Ambiente de execucao fora das tres rotas oficiais da disciplina (WSL2, VirtualBox, Ubuntu nativo). | Medio   | Rota adotada registrada em docs/deciso.es.md e declarada no relatorio. Migracao para VirtualBox com a ISO fornecida planejada antes do TP2.         |
| R2 | Memória insuficiente para treinar o modelo localmente.                                             | Alto    | Treinamento realizado em nuvem com GPU gratuita; apenas o modelo exportado é executado na VM.                                                       |
| R3 | A webcam USB não é reconhecida dentro da máquina virtual.                                          | Alto    | Configurar passthrough USB e validar com v4l2-ctl antes do TP3; fallback documentado: publicação de vídeo pré-gravado pelo mesmo nó, via parâmetro. |
| R4 | Dataset pequeno, desbalanceado ou com imagens de baixa qualidade.                                  | Médio   | Transfer learning sobre rede pré-treinada; aumento de dados; coleta iniciada no TP2; espécies escolhidas por separabilidade visual.                 |
| R5 | Confusao entre especies                                                                            | Medio   | Matriz de confusao exigida pelo G4.6 como                                                                                                           |

| ID  | Risco                                                                   | Impacto | Mitigação                                                                                                                                 |
|-----|-------------------------------------------------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------|
|     | visualmente proximas.                                                   |         | metrica de acompanhamento; exibicao das tres predicoes mais provaveis; limiar de confianca com rejeicao explicita.                        |
| R9  | Arvore TF quebrada inviabilizando SLAM e Nav2 (TP3 e TP4).              | Alto    | Validar o G3.2 com tf2_tools view_frames antes de tocar no SLAM. Noventa por cento dos problemas de Nav2 sao problemas de TF disfarçados. |
| R10 | Treinamento do PPO (300k steps) iniciado tarde demais no TP5.           | Alto    | Disparar o treino no inicio da janela, em segundo plano, com checkpoints salvos, enquanto as demais atividades correm.                    |
| R11 | Trabalho perdido por falta de push, ou branch não mergeada na main.     | Medio   | Push ao fim de cada sessao; merge de dev para main antes de cada tag. Correcao olha exclusivamente a main.                                |
| R6  | Raspberry Pi 3 inoperante ou com desempenho insuficiente.               | Baixo   | A placa é item opcional; o escopo obrigatório é atendido integralmente pela webcam USB.                                                   |
| R7  | Conflito entre a carga de trabalho profissional e os prazos de entrega. | Médio   | Cronograma dimensionado para janelas noturnas; folga de uma semana antes de cada entrega; entregas parciais versionadas no GitHub.        |
| R8  | Perda de trabalho por corrupção da VM.                                  | Médio   | Commits frequentes no GitHub; snapshot da VM após cada marco concluído.                                                                   |

## 12. Critérios de Aceite

O projeto será considerado bem-sucedido se, ao final do TP5, os seguintes critérios forem verificáveis por demonstração:

| ID   | Critério                                                                       | Verificação          |
|------|--------------------------------------------------------------------------------|----------------------|
| CA01 | O comando ros2 doctor executa sem erros críticos no ambiente configurado.      | Captura de tela      |
| CA02 | Taxa estável por 10 s medida em /vision/contagem, a regua leve do pipeline.    | G1.2                 |
| CA03 | A segmentação HSV produz máscara coerente e contagem consistente do domínio.   | G1.3                 |
| CA04 | O serviço /vision/status responde corretamente via ros2 service call.          | Demonstração ao vivo |
| CA05 | A detecção de faces e a extração de features operam sobre os frames recebidos. | Demonstração ao vivo |
| CA06 | O grafo de comunicação exibe todos os nós e tópicos previstos na arquitetura.  | Saída do rqt_graph   |
| CA07 | CNN treinada com no mínimo 500 frames, comparada com e sem augmentation.       | G4.6                 |

| ID   | Critério                                                                               | Verificação          |
|------|----------------------------------------------------------------------------------------|----------------------|
| CA08 | A identificação retorna a taxonomia completa e as características da espécie.          | Demonstração ao vivo |
| CA09 | reproduzir.sh executa num clone limpo sem que o avaliador precise adivinhar comandos.  | GF.0                 |
| CA10 | Todos os artefatos entregues no Moodle: PDF, ZIP, link do repositório e link do vídeo. | Checklist            |

## 13. Premissas e Restrições

### 13.1. Premissas

- A ISO Ubuntu 22.04 fornecida pelo professor está íntegra e contém ou permite a instalação da distribuição ROS 2 Humble.
- Existem imagens de licença aberta em quantidade suficiente para as quatro espécies selecionadas.
- O hardware do notebook suporta virtualização assistida por hardware, habilitada na BIOS/UEFI.

### 13.2. Restrições

- 8 GB de RAM no host, limitando a alocação para a VM e inviabilizando treinamento local.
- Ausência de GPU dedicada disponível para o ambiente virtualizado.
- Disponibilidade concentrada em período noturno e fins de semana.
- Uso obrigatório de câmera e de ROS 2, conforme exigência da disciplina.
- Vídeo de defesa limitado a 5 minutos, com tela e webcam simultâneas.

## 14. Partes Interessadas

| Parte                | Papel                          | Interesse                                                                        |
|----------------------|--------------------------------|----------------------------------------------------------------------------------|
| Aluno                | Executor e responsável técnico | Concluir as cinco entregas com qualidade e domínio demonstrável do conteúdo      |
| Professor orientador | Aprovador e avaliador          | Verificar aderência ao enunciado, profundidade técnica e clareza da documentação |
| Instituto Infnet     | Instituição                    | Arquivamento institucional e conformidade acadêmica                              |

## 15. Aprovação

Este Termo de Abertura formaliza o início do projeto ARACNE e estabelece escopo, objetivos, arquitetura, cronograma e critérios de aceite acordados para as entregas TP1 a TP5.

Aluno responsável — [seu nome completo]

---

Professor orientador — [nome do professor]

Data: \_\_\_\_ / \_\_\_\_ / \_\_\_\_